

Arrays in C

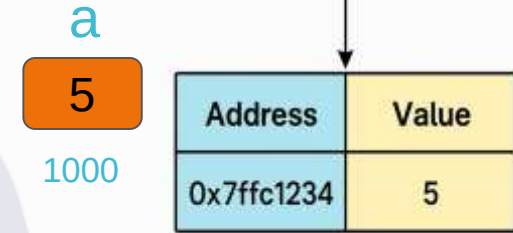
Why need of array?

Example:

```
Int a = 5;
```

```
Int a = 6;
```

To process large amount of data.



Suppose you want to process any number what you will need to do ?
You have to declare a variable then value assign to it

Example : To store roll numbers of 60 students

In one variable store one roll number

```
int a = 10;
```

```
int b = 40;
```

```
int c = 50;
```

```
.....
```

```
.....
```

```
.....
```

`a`
10

1000

`b`
40

2000

Under one variable name only can store 60 roll numbers.

```
int roll_no [60];
```

4 X 60 = 240 bytes allocated in consecutive memory locations.

What is an Array

An array in C is a collection of data items of similar data type stored at contiguous memory locations.

Core Concepts:

- **Homogeneous:** Every element in the array must be of the exact same data type (e.g., all int, all float, or all char).
 - `int a[5];`
 - `float arr[10];`
 - `char arr[10];`
- **Contiguous Memory:** Elements are placed right next to each other in system memory, enabling rapid calculation of memory addresses.
- **Fixed Size:** Once declared, the size of a standard array cannot be changed during runtime.
- **0-Based Indexing:** The first item is always located at index 0, and the last item is located at index size - 1.

Declaring and Initializing Arrays

Declaration: You can declare an array by stating its data type, name, and size inside square brackets [].

`data_type array_name[size];`

Example: `int arr[5];`

Initialization:

During Declaration: `int arr[5] = {1, 2, 3, 4, 5};` or

After Declaration:

`arr[0] = 1;`

`arr[1] = 2;`

Default Values:

- Global arrays are initialized to 0 by default.
- Local arrays contain garbage values.

Valid or Invalid?

`Int a[];`

Invalid – Missing Size creates an incomplete type.
Requires an initializer to be valid.

`Int a[-5];`

Invalid – Array size must be greater than zero.
Negative dimensions are illegal.

`Int a[2+2];`

Valid – Evaluates to a size of 4.
Constant expressions are valid

`Int a[2*5];`

Valid – Evaluates to a size of 10.
Multiplication yields a valid constant.

`Int a[11/5];`

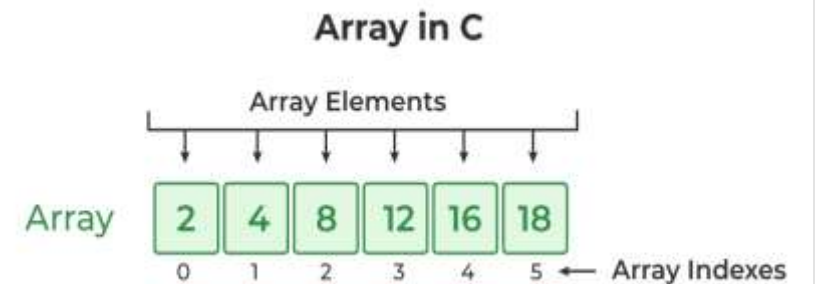
Valid – Evaluates to 2 via integer division.
Truncated whole numbers are valid sizes.

Accessing Array Elements

- ❖ Use the index to access elements:
- ❖ `Array_name[index];`
- ❖ Array index is always started from 0.
- ❖ Random access.

Example:

```
int arr[6] = {2, 4, 8, 12, 16, 18};  
printf("%d", arr[1]); // Output: 4
```



Length of Array in C

- ❖ The **maximum number of elements that an array can hold.**
- ❖ It is also known as the size of an array that is used to determine the **memory required to store all of its elements.**

```
#include <stdio.h>

int main() {
    int arr[50] = { 1, 2, 3, 4, 5 };
    // Find the size of the array arr
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("%d", n);
    return 0;
}
```

output : 50

SizeOf()

`sizeof(numbers)`

This gives the **total size in bytes of the entire array**.

For example, `int arr[5];`

`sizeof(arr) = 50 * 4 = 200 bytes`

`sizeof(arr[0])`

This gives the size in bytes of **the first element** (which is the size of one `int`, usually 4 bytes).

❓ **So when you divide:**

`sizeof(numbers) / sizeof(numbers[0])`

`200 / 4 = 50`

This tells you: **There are 50 elements in the array.**

Fixed Size

Random Access

**Homogeneous
Data**

**Multidimensional
Arrays**

Properties of an Array

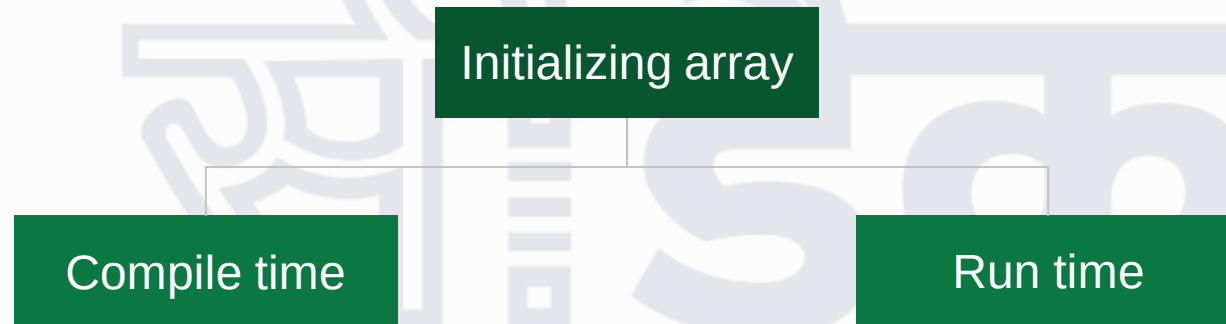
Indexing

No Bound checking

**Contiguous Memory
Allocation**

**Pointer
Relationship**

Initializing Array



Compile-time initialization

It means the values are given to the array **before the program starts running**, and the compiler knows those values while compiling the program.

`Int a[5] = {0,11,2,324,2};` -Creates an array named a of size 5 and fills all 5 slots with the specified numbers.

`Int a[] = {1,2,23,50};` - Declares an array without specifying the size inside the brackets []. The compiler automatically calculates the size based on the number of elements provided (here, size is 4).

`Int a[5]={1,2,3};` remaining two locations would be zero. -Partial initialization.

`Int a[5];` -Declares an array of size 5 but does not initialize it. The array contains **garbage values**

`Int a[2] = {2,3,4,5};` - **Error**. You allocated space for only 2 elements but tried to store 5 elements. This causes a compile-time "excess elements in array initializer" error.

`Int a[2]={};` - **Error**. Standard C programming (ISO C) does not allow empty initialization blocks without giving a size or at least one element.

Compile-time initialization

It means the values are given to the array **before the program starts running**, and the compiler knows those values while compiling the program.

```
#include <stdio.h>
```

```
int main() {  
    int numbers[] = {10, 20, 30, 40, 50}; // initialization  
    int length = sizeof(numbers) / sizeof(numbers[0]);  
  
    for (int i = 0; i < length; i++) {  
        printf("%d\n", numbers[i]); // print each element  
    }  
  
    return 0;  
}
```

- **numbers[]** → C automatically determines the array size from the initializer.
- **sizeof(numbers) / sizeof(numbers[0])** → finds the number of elements in the array.
- **numbers[i]** → accesses each element using its index.

Runtime initialization of array

Run-time initialization means the values are obtained while the program is running. Usually, we get values from the user using `scanf()`.

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter array size: ");
    scanf("%d", &n);
    int arr[n]; // array size decided at runtime
    // Input values
    for (int i = 0; i < n; i++) {
        printf("Enter element %d: ", i);
        scanf("%d", &arr[i]);
    }
    // Print values
    printf("You entered:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

```
Enter array size: 5
Enter element 0: 23
Enter element 1: 45
Enter element 2: 5
Enter element 3: 6
Enter element 4: 7
You entered:
23 45 5 6 7
```



MCQs

Which of the following array declarations is correct in C?

- a) `int numbers[5] = {1, 2, 3, 4, 5};`
- b) `float rates[] = {2.5, 3.0, 4.5, 5.5};`
- c) `char name[10] = "John";`
- d) All of the above

d) All of the above



MCQs

How do you access the third element in an array named 'data' in C?

- a) data[2]
- b) data[3]
- c) data(2)
- d) data(3)

a) data[2]



MCQs

What is the default value of elements in an uninitialized array in C?

- a) Null
- b) 0
- c) Garbage values
- d) It depends on the compiler

c) Garbage values



MCQs

What is the size of an array 'arr' declared as `int arr[10]` in C?

- a) 10 bytes
- b) 20 bytes
- c) 40 bytes
- d) Depends on the system

d) Depends on the system



MCQs

What is the output of the following C code?

```
int arr[] = {1, 2, 3, 4, 5};  
printf("%d", arr[3]);
```

- a) 3
- b) 4
- c) 5
- d) Error

b) 4



MCQs

Which of the following is a correct way to initialize an array in C?

- a) `int arr[3] = {0};`
- b) `int arr[3] = {1, 2, 3, 4};`
- c) `int arr[] = {1, 2, 3, 4};`
- d) a) and c)

d) a) and c)



MCQs

Can the size of an array be changed at runtime in C?

- a) Yes, always
- b) No, never
- c) Only for dynamic arrays
- d) Only if it is a global array

b) No, never



MCQs

What is the output of the following C code?

```
int arr[3] = {0};  
printf("%d %d %d", arr[0], arr[1],  
arr[2]);
```

- a) 0 0 0
- b) Garbage values
- c) Error
- d) 0 followed by garbage values

a) 0 0 0



MCQs

Which of the following statements about arrays in C is correct?

- a) Arrays can store elements of different data types
- b) Array indices start at 1
- c) The size of an array must be a constant expression
- d) Elements in an array are accessed using parentheses

c) The size of an array must be a constant expression



MCQs

How can the number of elements in an array be determined in C?

- a) Using the sizeof operator
- b) Using the length() function
- c) By manually counting the elements
- d) Using the size() function

a) Using the sizeof operator

Task

1) Write a C program to input an array from the user, then print the original and reversed array.

Sample Output:

Enter the number of elements: 5

Enter 5 elements:

1 2 3 4 5

Original array:

1 2 3 4 5

Reversed array:

5 4 3 2 1

Solution

Write a C program to input an array from the user, then print the original and reversed array.

```
#include <stdio.h>

int main() {
    int n;

    // Take size from user
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n]; // Declare array of size n

    // Take array elements from user
    printf("Enter %d elements:\n", n);
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Print original array
    printf("Original array:\n");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Print reversed array
    printf("Reversed array:\n");
    for(int i = n - 1; i >= 0; i--) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Task

2) WAP to enter marks of 5 subjects, then calculate and print the total sum and average using array.

Sample Output:

Enter marks of 5 subjects:

80 75 90 85 70

Total Marks = 400

Average Marks = 80.00

Solution

Enter marks of 5 subjects, then calculate and print the total sum and average using array.

```
#include <stdio.h>
int main() {
    int marks[5], sum = 0;
    float average;

    printf("Enter marks of 5 subjects:\n");
    for(int i = 0; i < 5; i++) {
        scanf("%d", &marks[i]);
        sum += marks[i];
    }
    average = sum / 5.0;
    printf("Total Marks = %d\n", sum);
    printf("Average Marks = %.2f\n", average);
    return 0;
}
```

Task

3) Write a C program to read an array of 10 integers and count the total number of even and odd elements.

Sample Output:

Enter 10 integers:

1 2 3 4 5 6 7 8 9 10

Total even numbers: 5

Total odd numbers: 5

Task

4) WAP that reads two arrays, calculates the sum of corresponding elements, and stores the result in a third array.

Sample Output:

Enter 5 elements for the first array:

1 2 3 4 5

Enter 5 elements for the second array:

6 7 8 9 10

Sum of the arrays:

7 9 11 13 15

Types of Arrays

One-Dimensional Array

A single row or list of elements.

Example: `int arr[5];`

Multi-Dimensional Array

Arrays with two or more dimensions (e.g., tables or matrices).

Example: `int matrix[3][4];`

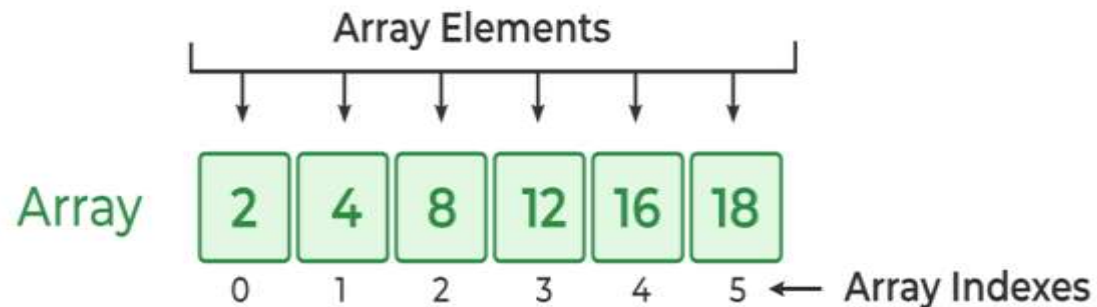
One-dimensional array

The One-dimensional arrays, also known as 1-D arrays.

A one-dimensional (1D) array is a linear list of data items of the same type stored in continuous memory. You access each item using a single index number that usually starts at zero.

Syntax of 1D Array in C

```
array_name [size];
```



Example of One Dimensional Array

```
#include <stdio.h>

int main()
{
    int arr[5]; // 1d array declaration
    // 1d array initialization using for loop
    for (int i = 0; i < 5; i++) {
        arr[i] = i;
    }

    printf("Elements of Array: ");
    // printing 1d array by
    // traversing using for loop
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

Elements of Array: 0 1 2 3 4

Multidimensional Array

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
```

	COLUMN 0	COLUMN 1	COLUMN 2
ROW 0	1	4	2
ROW 1	3	6	8

Multidimensional Arrays in C

- A multi-dimensional array can be defined as an array that has more than one dimension.
- Array of arrays.
- Some of the popular multidimensional arrays are 2D arrays and 3D arrays.

Syntax of 2D Array in C

`array_name[size1][size2];`

Here,

size1: Size of the first dimension.

size2: Size of the second dimension.

2D Array

1	2	3	4
1	2	3	4
1	2	3	4
1	2	3	4

Multidimensional Arrays in C

Example

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };  
printf("%d", matrix[0][2]); // Outputs 2
```

Example of 2D Array in C

```
#include <stdio.h>

int main()
{
    // declaring and initializing 2d array
    int arr[2][3] = { 10, 20, 30, 40, 50, 60 };
    printf("2D Array:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output

2D Array:
10 20 30
40 50 60

Example of 3D Array in C

```
#include <stdio.h>

int main()
{
    // 3D array declaration
    int arr[2][2][2] = { 10, 20, 30, 40, 50, 60 };
    // printing elements
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                printf("%d ", arr[i][j][k]);
            }
            printf("\n");
        }
        printf("\n \n");
    }
    return 0;
}
```

Output

10 20

30 40

50 60

0 0

Example of 3D Array in C

```
#include <stdio.h>

int main() {
    int arr[2][3][2]; // arr[x][y][z]
    int i, j, k;
    // Taking input
    printf("Enter values for a 3D array (2 x 3 x 2):\n");
    for (i = 0; i < 2; i++) { // First dimension
        for (j = 0; j < 3; j++) { // Second dimension
            for (k = 0; k < 2; k++) { // Third dimension
                printf("Enter value for arr[%d][%d][%d]:", i, j, k);
                scanf("%d", &arr[i][j][k]);
            }
        }
    }

    // Displaying output
    printf("\nYou entered:\n");
    for (i = 0; i < 2; i++) {
        for (j = 0; j < 3; j++) {
            for (k = 0; k < 2; k++) {
                printf("arr[%d][%d][%d] = %d\n", i, j, k, arr[i][j][k]);
            }
        }
    }
    return 0;
}
```

Operations on Arrays :

1.Traversing

Access each element using a loop.

```
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int N = sizeof(arr) / sizeof(arr[0]);
    // Traverse and print elements using a for loop
    printf("Array elements using loop: ");
    for (int i = 0; i < N; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

Output : Array elements using
loop: 1 2 3 4 5

Operations on Arrays :

2.Insertion

Add an element at a specific position.

```
#include <stdio.h>

int main() {

    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49,
51};

    num[2] = 100;

    for(int i = 0; i < 10; i++) {
        printf("%d\n", num[i]);
    }

    return 0;
}
```

Output : 50 55 100 73 45 21 39
70 49 51

Operations on Arrays : 3.Search

Find an element by value

```
#include <stdio.h>

int main() {

int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49,
51};

for(int i = 0; i < 10; i++) {
    if(num[i] == 21) {
        printf("Found at index %d", i);
    }
}

return 0;
}
```

Output: Found at index 5

Operations on Arrays :

4.Deletion

Set the value at a specific index to 0 or shift elements.

```
#include <stdio.h>

int main() {
    int arr[100] = {2, 4, 6, 8, 10}; // Initial array
    int size = 5; // Current size of the array
    int positionToDelete ; // Index of the element to delete

    printf("Enter the position of an element to delete : ");
    scanf(" %d",&positionToDelete);

    for (int i = positionToDelete; i < size - 1; i++)
    {
        arr[i] = arr[i + 1];
    }
    size--; // Update the size of the array
    for (int i = 0; i < size; i++) { // Print the updated array
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Output : Enter the position of an element to delete : 2
2 4 8 10

Practice

- Write a C program to insert an element at a specified position.
- Search for a given value in an array using linear search; print its index or "Not found."
- Find the largest element in an array.
- Count how many even and how many odd numbers are in an array.
- Write a program to reverse an array
- Delete a given value from an array